

CIERRE TECNICO - MODULO 1 - SPRINT 4

Equipo

Modulo 1: Modelo + Tablero

Integrantes: Nicolas Pardo, Alfa Umara Jalo y Juan Pablo Gonzalez.

Scrum Master del modulo: Nicolas Pardo.

Objetivo del sprint

En el Sprint 4 el Modulo 1 tenia que cerrar el modelo de dominio para que una ciudad pudiera guardarse y recuperarse sin perder informacion importante. La meta era que una ciudad recuperada desde persistencia pudiera usarse igual que una ciudad creada en memoria.

El Modulo 1 no implementa JDBC. Esa responsabilidad corresponde al Modulo 4. Desde este modulo se entrega la API necesaria para que persistencia pueda reconstruir objetos del dominio sin saltarse validaciones internas.

Trabajo de Juan Pablo: reconstruccion de Ciudad

Se agrego en Ciudad el metodo:

```
public static Ciudad reconstruir(String nombre, int filas, int columnas, int expansionesRealizadas)
```

Este metodo permite crear una instancia valida desde datos persistidos. La reconstruccion reutiliza las validaciones del constructor normal para nombre, filas y columnas. Tambien valida que expansionesRealizadas no sea negativo y no supere el maximo permitido.

Despues de reconstruir, la ciudad queda con tablero inicializado, tipo estructural recalculado segun filas y columnas, contador de expansiones conservado y lista para repoblarse con bloques desde persistencia.

Archivo principal modificado:

```
src/main/java/org/synthcity/modulo_1/Ciudad.java
```

Trabajo de Alfa: factoria de Bloques

Se agregaron en Bloque los metodos:

```
public static Bloque crearBloque(TipoBloque tipo, Posicion posicion)
public static Bloque crearBloque(TipoBloque tipo, Posicion posicion, boolean activo)
```

La primera version crea el bloque concreto segun su TipoBloque. La segunda version permite reconstruir tambien el estado activo o inactivo guardado en base de datos.

Mapeo implementado:

- RESIDENCIAL crea BloqueResidencial
- ENERGIA crea BloqueEnergia
- INDUSTRIAL crea BloqueIndustrial
- SERVICIOS crea BloqueServicios
- TRANSPORTE crea BloqueTransporte

Archivo principal modificado:

src/main/java/org/synthcity/modulo_1/bloques/Bloque.java

Trabajo de Nicolas: cierre y contrato de persistencia

Como parte del cierre tecnico y la coordinacion del modulo, se agrego una clase de validacion del contrato de persistencia:

ContratoPersistenciaModulo1

Su objetivo es revisar que una Ciudad y sus Bloques tengan los datos minimos necesarios antes de guardarse o integrarse con persistencia.

La clase valida:

- Ciudad no nula
- Nombre de ciudad presente
- Dimensiones validas
- Capacidad coherente con filas y columnas
- Tipo estructural presente
- Expansiones realizadas no negativas
- Lista de bloques no nula
- Bloques no nulos
- Tipo de bloque presente
- Posicion de bloque presente
- Bloques dentro de los limites de la ciudad

Archivos principales modificados:

src/main/java/org/synthcity/modulo_1/ContratoPersistenciaModulo1.java

src/test/java/org/synthcity/modulo_1/ContratoPersistenciaModulo1Test.java

API util de Ciudad

- getNombre()
- getFilas()
- getColumnas()
- getCapacidadMaxima()
- getTipoEstructural()
- getExpansionesRealizadas()
- listarBloques()
- listarBloquesActivos()
- addBloque(Bloque bloque)

API util de Bloque y Posicion

De Bloque:

- getTipo()
- getPosicion()
- estaActivo()
- activar()
- desactivar()
- cambiarEstado(boolean estado)

De Posicion:

- getFila()
- getColumna()

Datos que debe persistir Modulo 4

Para reconstruir correctamente una ciudad, el Modulo 4 debe guardar al menos:

- Nombre de la ciudad
- Filas
- Columnas
- Expansiones realizadas
- Bloques de la ciudad
- Tipo de cada bloque
- Fila de cada bloque
- Columna de cada bloque
- Estado activo o inactivo de cada bloque

El tipo estructural puede guardarse como dato de consulta, pero el modelo lo recalcula desde las dimensiones actuales para mantener coherencia.

Pruebas agregadas

En CiudadTest se agregaron pruebas para:

- Reconstruccion con dimensiones validas
- Reconstruccion con cero expansiones
- Reconstruccion con expansiones mayores que cero
- Nombre invalido
- Dimensiones invalidas
- Dimensiones por encima del maximo
- Expansiones negativas
- Expansiones por encima del maximo permitido
- Tipo estructural igual al de una ciudad normal con las mismas dimensiones

En BloqueTest se agregaron pruebas para:

- Creacion por factoria de los cinco tipos de bloque
- Conservacion de tipo y posicion
- Validacion de tipo nulo
- Validacion de posicion nula
- Reconstruccion con estado activo
- Reconstruccion con estado inactivo
- Comparacion del comportamiento funcional con los constructores directos

En ContratoPersistenciaModulo1Test se agregaron pruebas para:

- Ciudad persistente con bloques activos e inactivos
- Rechazo de ciudad nula
- Rechazo de bloque nulo
- Rechazo de bloque fuera de los limites de la ciudad

Ramas de trabajo

Rama de Juan Pablo:

feature/modulo1-juanpablo-reconstruccion-ciudad

Rama de Alfa:

feature/modulo1-alfa-factoria-bloques

Rama de Nicolas:

feature/modulo1-nico-documentacion-integracion

Orden recomendado de merge

1. feature/modulo1-juanpablo-reconstruccion-ciudad
2. feature/modulo1-alfa-factoria-bloques
3. feature/modulo1-nico-documentacion-integracion

Este orden permite integrar primero la reconstruccion de ciudad, despues la reconstruccion de bloques y al final el contrato de cierre del modulo.

Estado de pruebas

Se dejo cobertura unitaria para los cambios del modelo. En este equipo queda pendiente ejecutar la suite completa con Maven desde un entorno donde mvn este disponible o desde IntelliJ.

Pendientes de integracion

- Modulo 4 debe usar Ciudad.reconstruir(...) para cargar ciudad.
- Modulo 4 debe usar Bloque.crearBloque(...) para cargar bloques.
- Se debe probar guardar, cerrar la aplicacion, cargar y volver a simular.
- Se debe comparar una ciudad original contra una ciudad reconstruida con los mismos datos.

Conclusion

El Modulo 1 queda preparado para el cierre del sistema. La persistencia puede reconstruir ciudad y bloques usando metodos publicos del dominio, manteniendo validaciones, dimensiones, tipo estructural, contador de expansiones y estado de bloques.